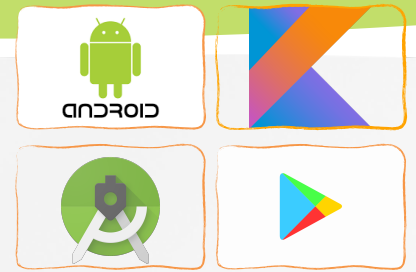




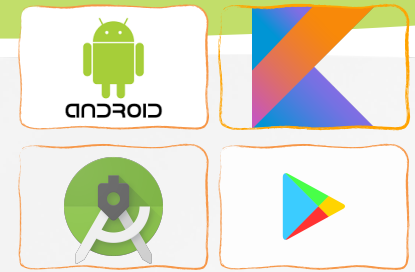
Software Development for Mobile Devices

Nguyen Le Hoang Dung
[*nlhdung@fit.hcmus.edu.vn*](mailto:nlhdung@fit.hcmus.edu.vn)



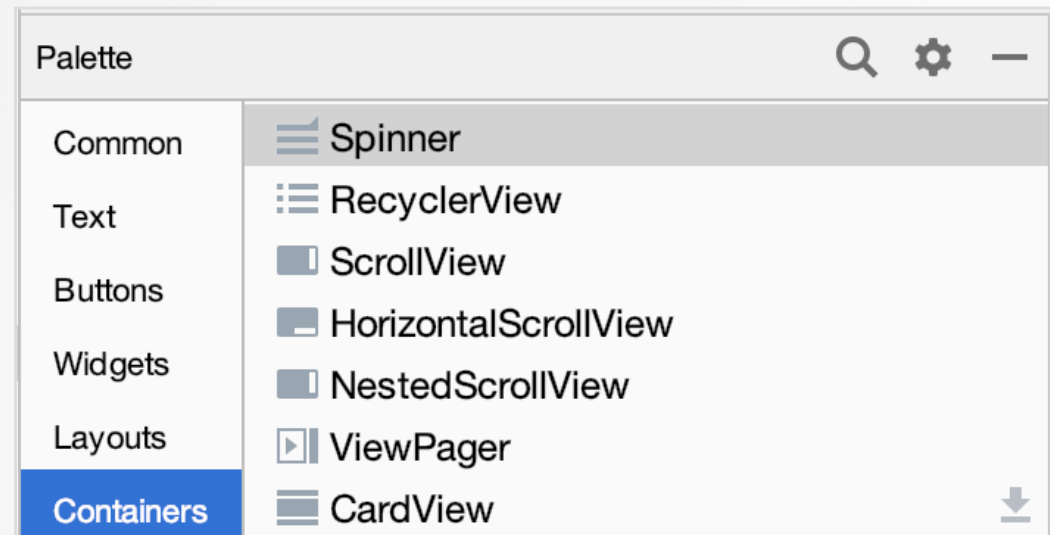
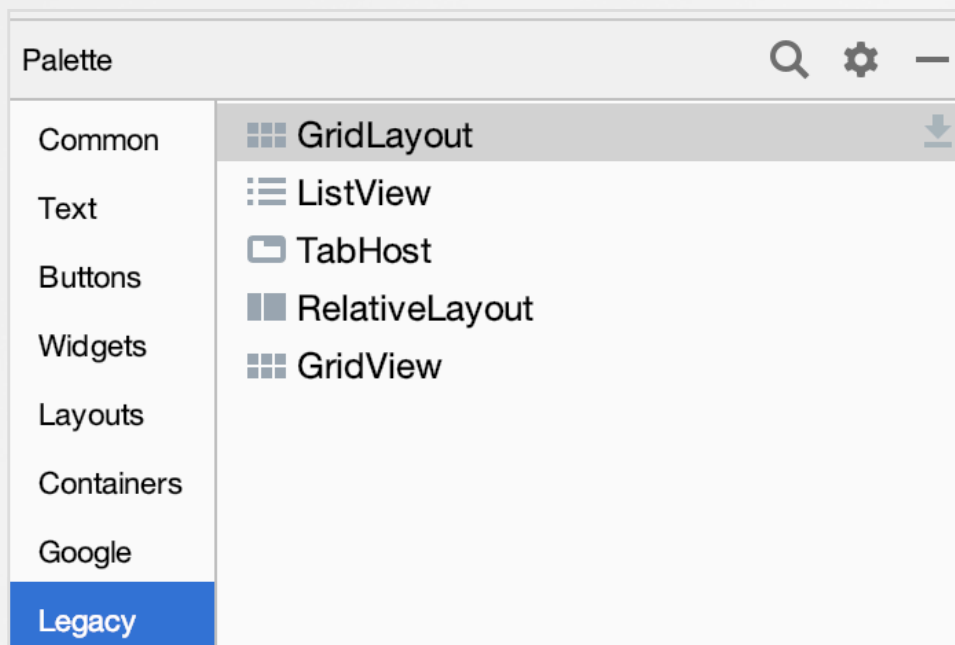
Contents

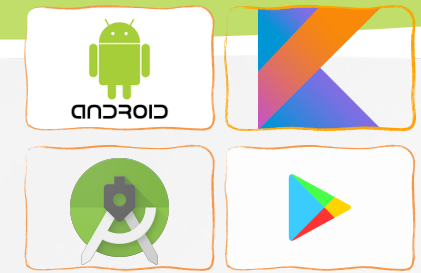
- List-Based Widgets:
 - ListViews and custom ListView
 - RecyclerView
 - Spinner,
 - GridView,
 - AutoComplete TextView
 - HorizontalScrollView



List-Based Widgets

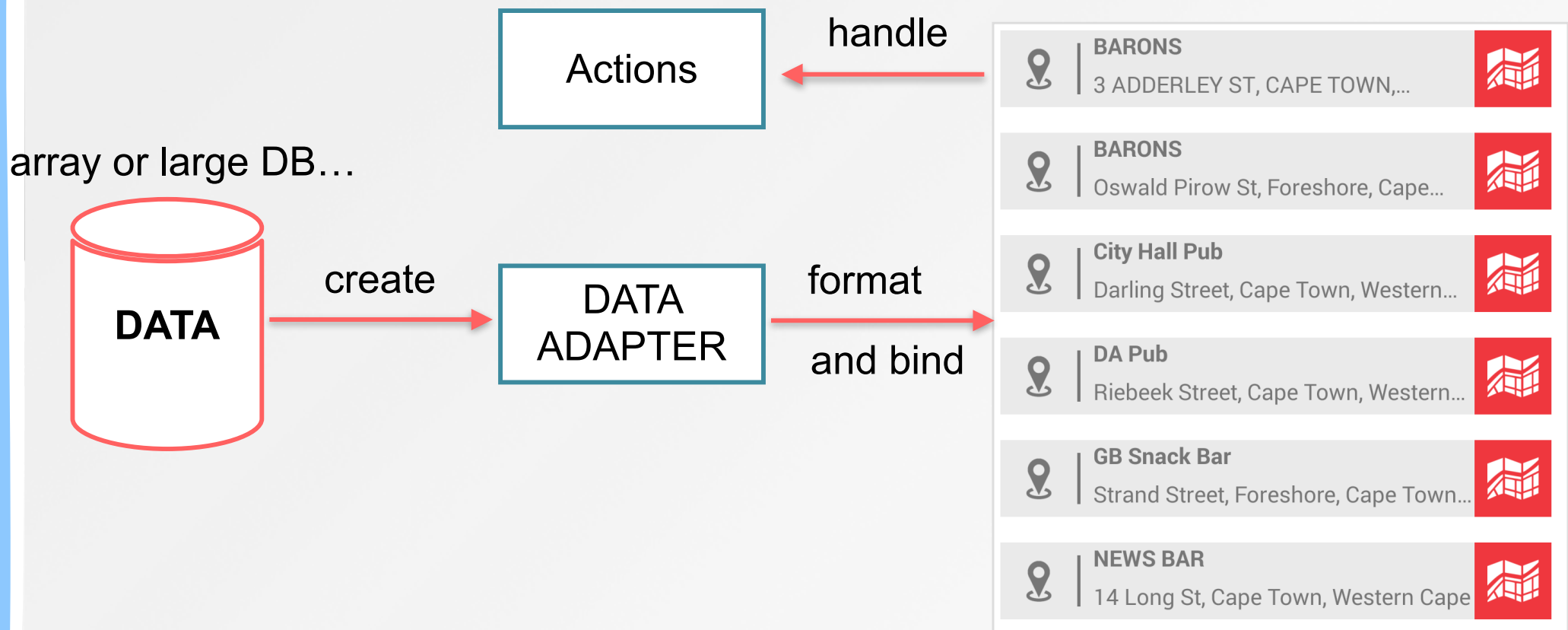
- List-Based Widgets:
 - Spinner,
 - ListViews,
 - RecyclerView
 - ScrollViews, HorizontalScrollView
 - GridView,
 - Image Gallery...

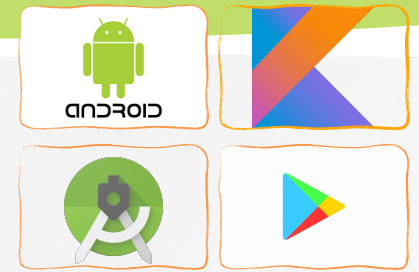




List-Based Widgets (cont.)

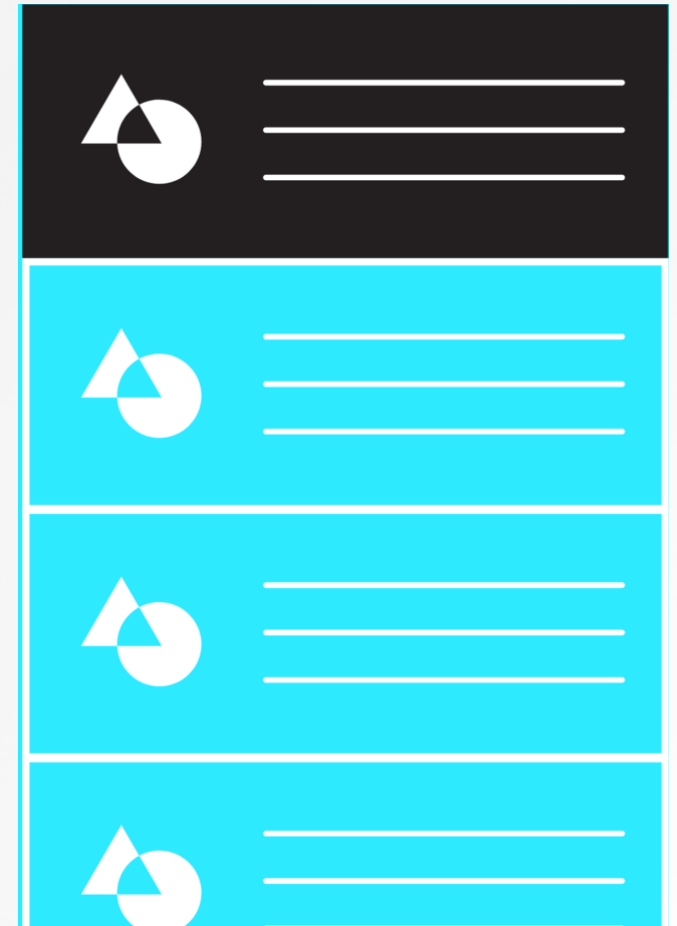
- Passing data to the visual representation

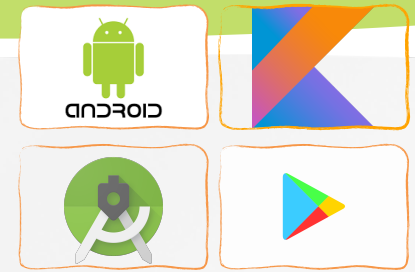




List-Based Widgets (cont.)

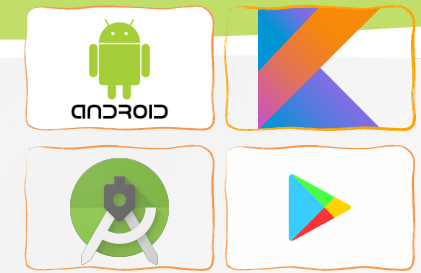
- The Android **ListView** widget is the most common UI used to display data supplied by a **data adapter**.
- ListViews are **scrollable**, all rows from large data sets have a chance to be shown.
- Users can **tap** on a row to make a selection.
- A row could display one or more lines of text as well as images.



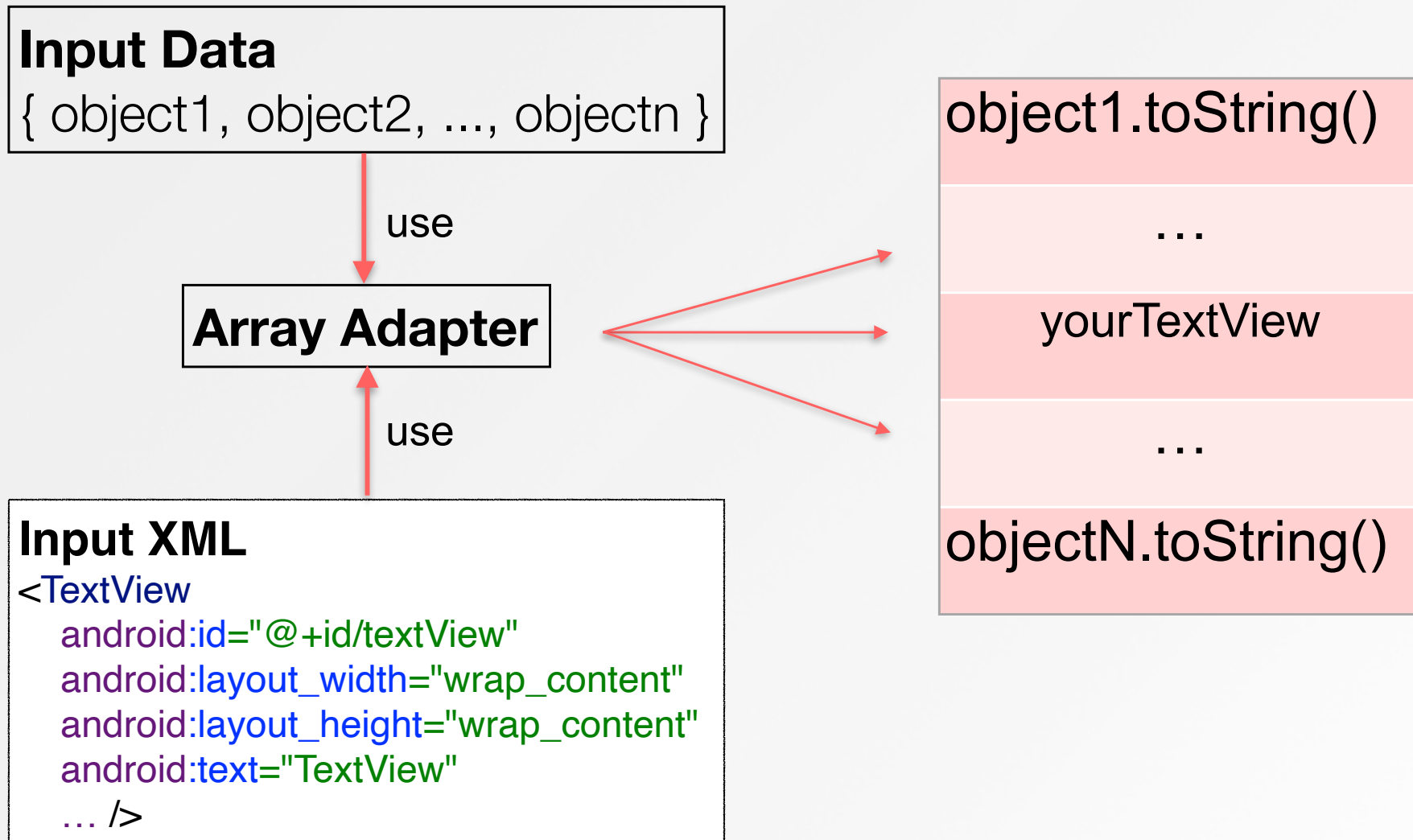


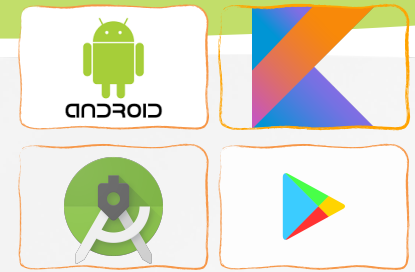
ArrayAdapter – A Data Beautifier

- An **`ArrayAdapter<T>`** is a concrete implementation of the (abstract) **`BaseAdapter`** class.
- Its mission
 - Accept an array of objects (of some arbitrary type `T`), individually process each object by applying its **`toString()`** method.
 - Move the formatted output string to a `TextView`. The formatting operation is guided by a supplied XML layout specification.
- The `ArrayAdapter<T>` is generally used to process `<String>` objects. For other than `String`-type you must override the `toString()` method of your objects to establish what text will be displayed for the item in the list.
- For `ListView`s showing complex arrangement of visual elements –such as text+images
 - => Provide a ***custom*** made ***adapter*** in which the **`getView(...)`** method explains how to manage the placement of each data fragment in the complex layout.



ArrayAdapter – A Data Beautifier (cont.)



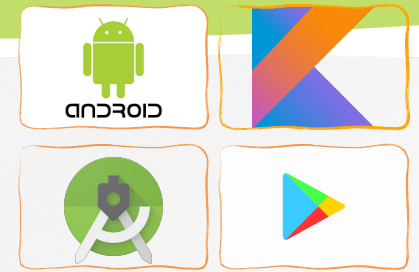


ArrayAdapter – A Data Beautifier (cont.)

- Simple usage of an ArrayAdapter<String>

```
val items = arrayOf(
    "Data-0", "Data-1", "Data-2",
    "Data-3", "Data-4", "Data-5",
    "Data-6", "Data-7", "Data-8")
val adapter = ArrayAdapter(this, R.layout.simple_list_item_1, items)
```

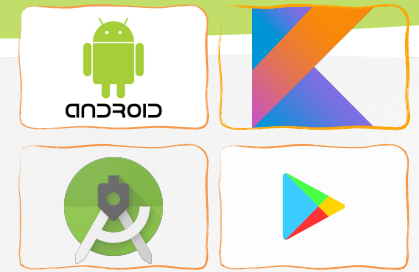
- Parameters:
 - *this*: the current activity's Context
 - *R.id.simple_list_item_1*: row layout including the TextView indicating how an individual row is written
 - *items*: the actual data source



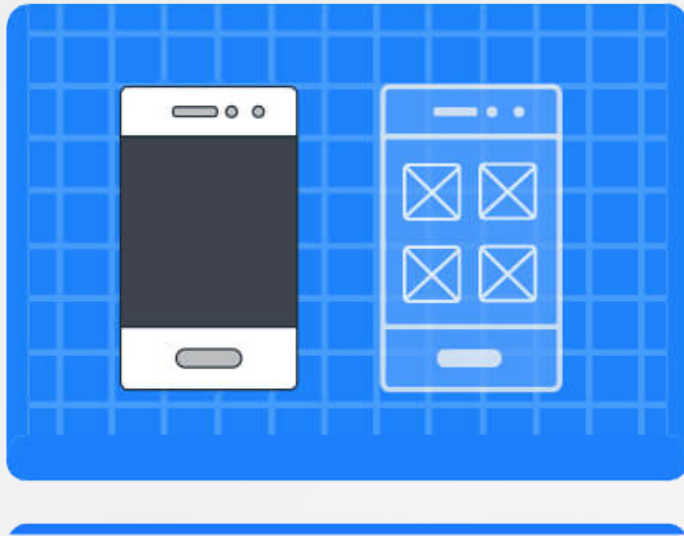
Using the ListView

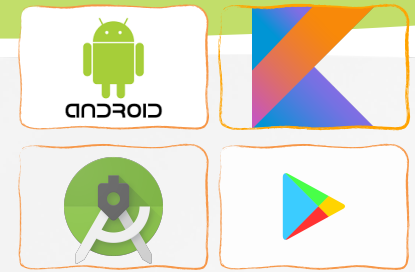
Key steps:

- Create ListView in layout XML
 - Create a custom row layout XML file to visualize the item if need
- Define a model class to use as the data source
- Add a ListView to your activity and to display the items
- Use basic ArrayAdapter to render the item
 - You can use your custom adapter
- Bind the adapter to the data source to populate the ListView



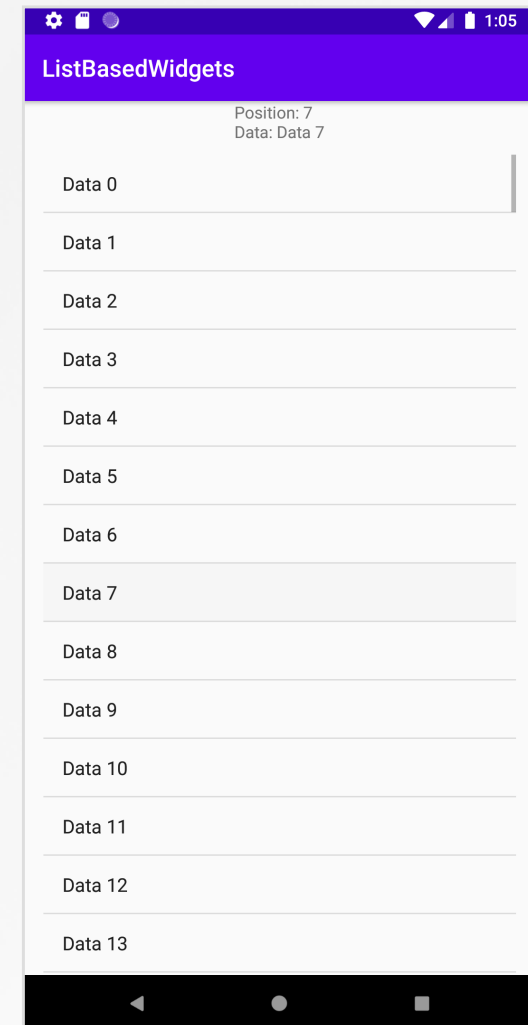
Example 08.1: Using the ArrayAdapter and ListView

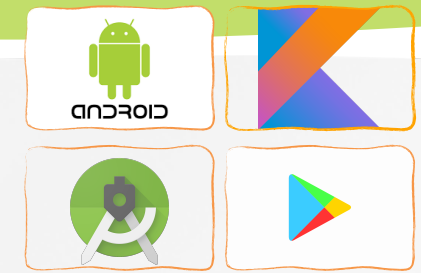




Example 08.1: Using the ArrayAdapter and ListView

- A simple collection of data items should be displayed in a ListView widget.
 - Data items are held in a `String[]` array.
 - Each row in the ListView shows a simple line of text taken from the array.
 - When the user makes a selection, we will display on a TextView the selected item.

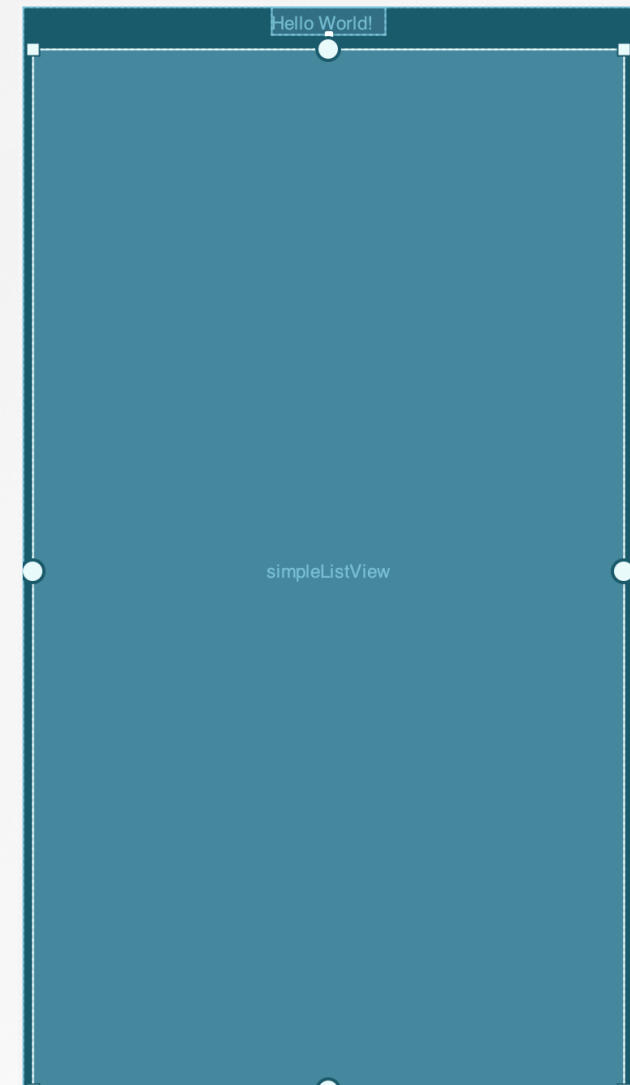


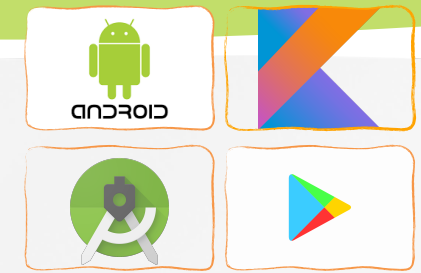


Example 08.1: XML layout

```
<?xml version="1.0" encoding="utf-8" ?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <TextView
        android:id="@+id/selectedTV"
        android:layout_width="wrap_content"
        android:layout_height="0dp"
        android:text="Hello World!"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <ListView
        android:id="@+id/simpleListView"
        android:layout_width="398dp"
        android:layout_height="0dp"
        android:layout_marginTop="10dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/selectedTV" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

built-in list layout
with empty data





Example 08.1:

MainActivity

```
var selectedTV: TextView? = null
var simpleListView: ListView? = null
var items = arrayListOf<String>()
```

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
```

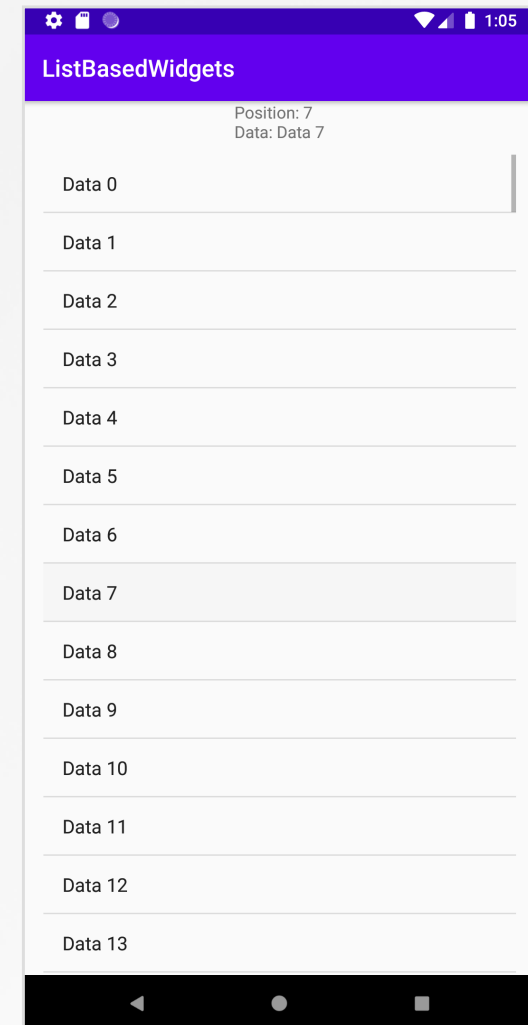
```
    simpleListView = findViewById(R.id.simpleListView)
    selectedTV = findViewById(R.id.selectedTV)
```

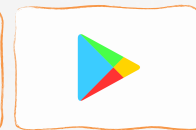
```
    //create simple data
```

```
    for (i in 0..200) {
        items.add("Data " + i.toString())
    }
```

```
    //...
```

```
}
```

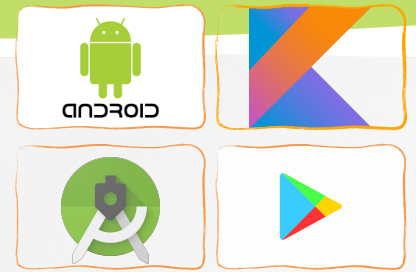




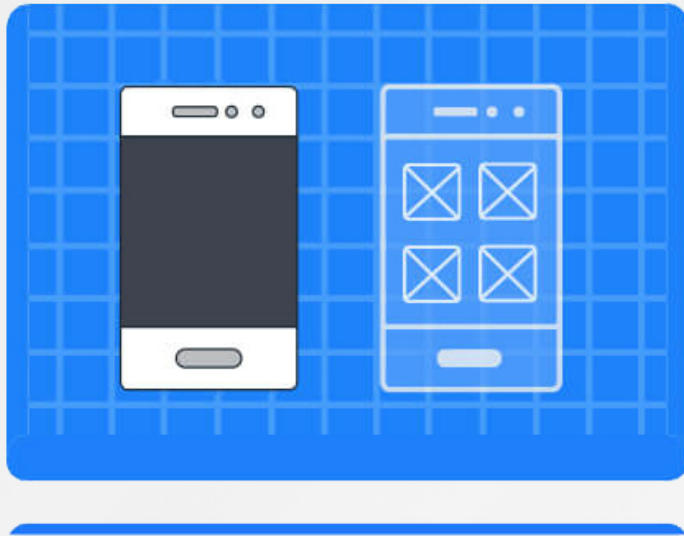
Example 08.1:

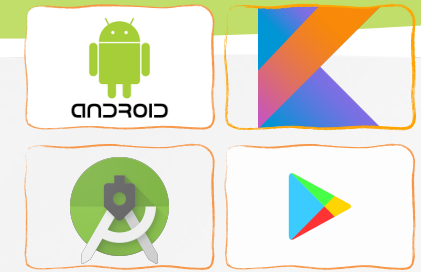
MainActivity

```
class MainActivity : AppCompatActivity() {  
    var selectedTV: TextView? = null  
    var simpleListView: ListView? = null  
    var items = arrayListOf<String>()  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        //...  
        val adapter = ArrayAdapter<String>(this, android.R.layout.simple_list_item_1, items)  
        simpleListView!!.adapter = adapter  
        simpleListView!!.setOnClickListener { adapterView, view, i, l ->  
            val text = "Position: " + i.toString() + "\nData: " + items[i]  
            selectedTV!!.setText(text)  
        }  
        //...  
    }  
}
```



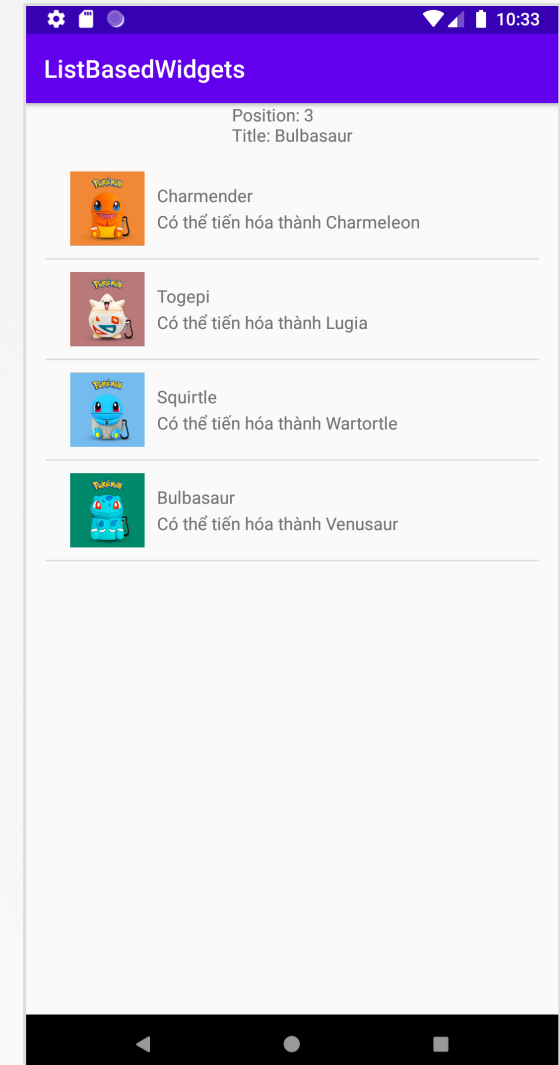
Example 08.2: Custom ListView

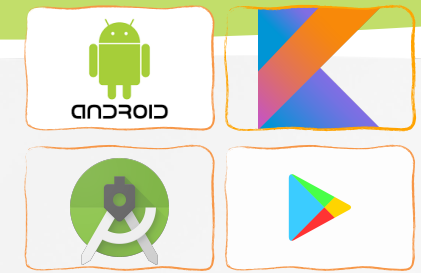




Example 08.2: Custom ListView

- Create XML layout for MainActivity
- Create XML layout for list item
- Create my adapter
- Edit your MainActivity

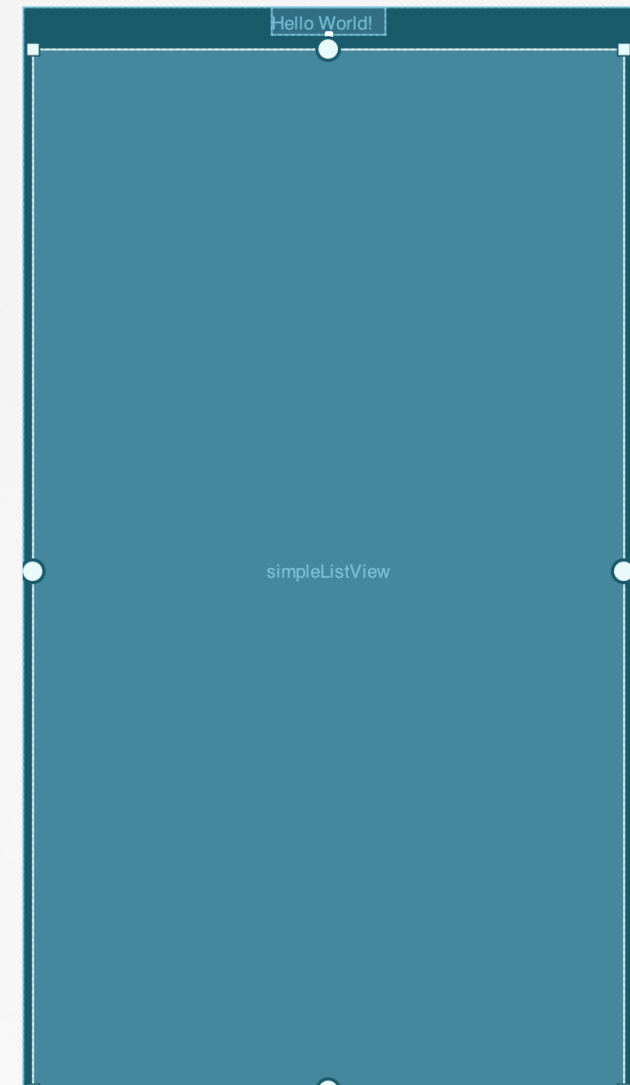


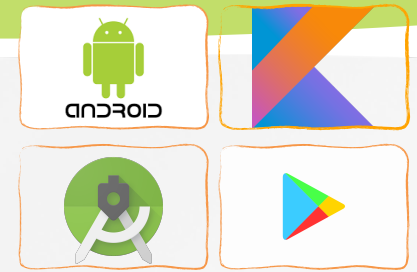


Example 08.2: Custom ListView XML layout for MainActivity

```
<?xml version="1.0" encoding="utf-8" ?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <TextView
        android:id="@+id/selectedTV"
        android:layout_width="wrap_content"
        android:layout_height="0dp"
        android:text="Hello World!"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <ListView
        android:id="@+id/customListView"
        android:layout_width="398dp"
        android:layout_height="0dp"
        android:layout_marginTop="10dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/selectedTV" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

built-in list layout
with empty data



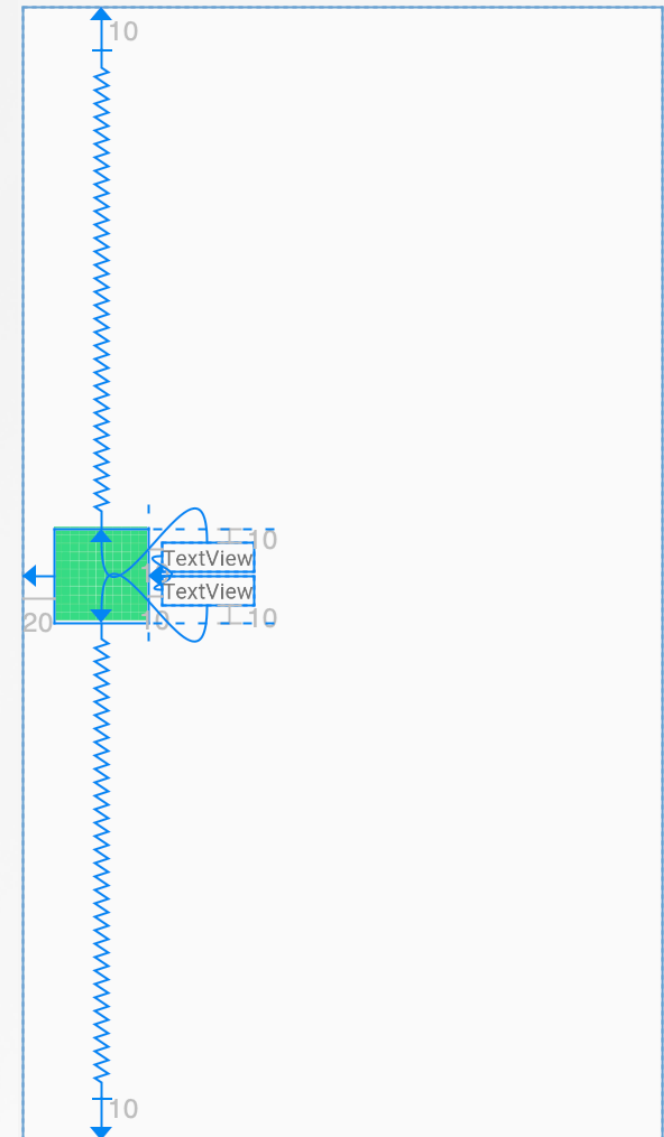


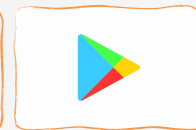
Example 08.2: Custom ListView XML layout for list item

```
<?xml version="1.0" encoding="utf-8" ?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ImageView
        android:id="@+id/iconIV"
        android:layout_width="60dp"
        android:layout_height="60dp"
        android:layout_marginStart="20dp"
        android:layout_marginTop="10dp"
        android:layout_marginBottom="10dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:srcCompat="@drawable/ic_launcher_background" />

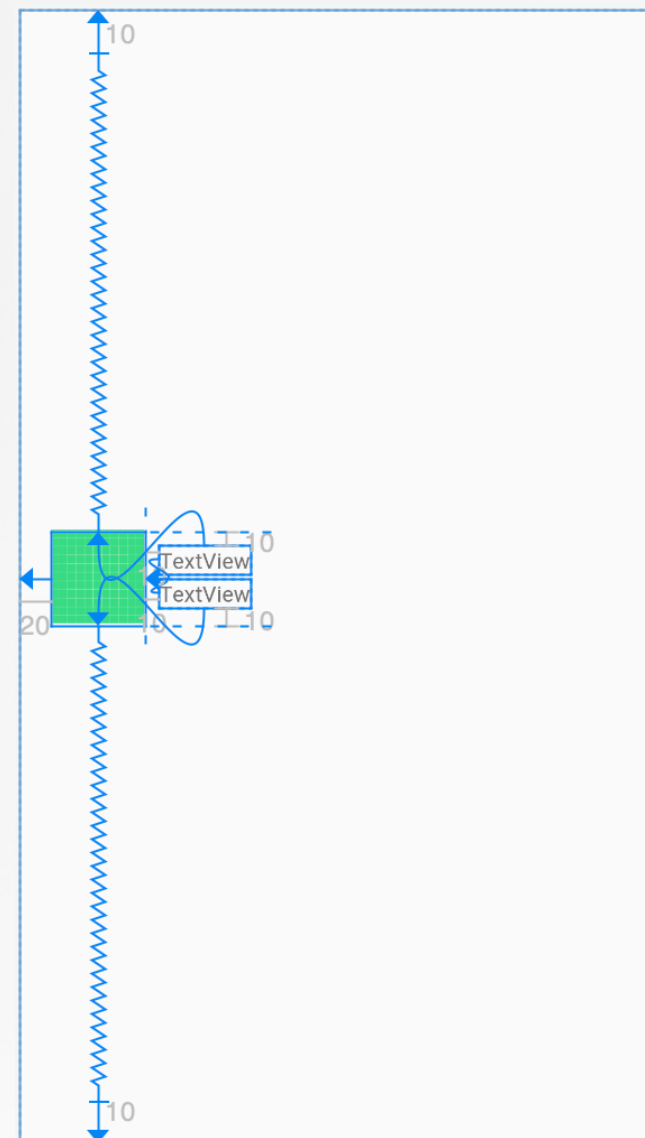
    <TextView
        android:id="@+id/subtitleTV"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="10dp"
        android:layout_marginBottom="10dp"
        android:text="TextView"
        app:layout_constraintBottom_toBottomOf="@+id/iconIV"
        app:layout_constraintStart_toEndOf="@+id/iconIV" />
```

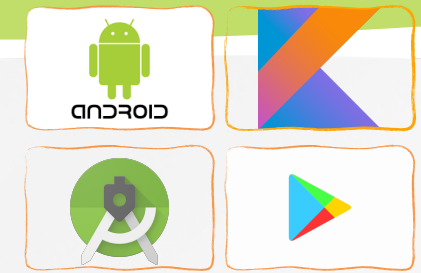




Example 08.2: Custom ListView XML layout for list item

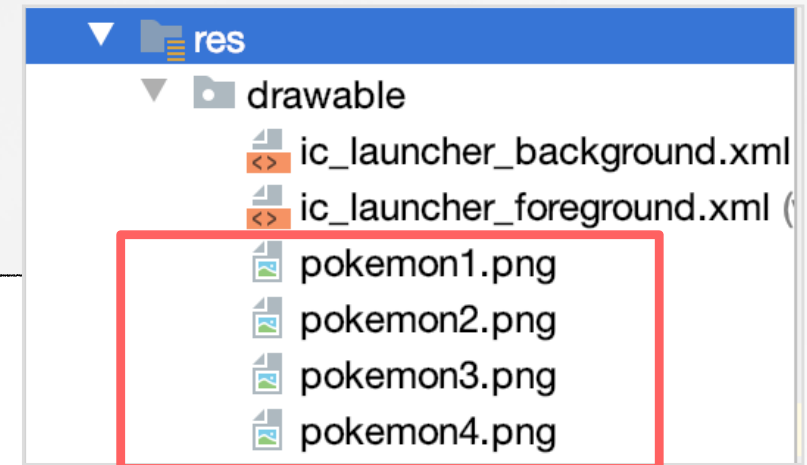
```
<TextView  
    android:id="@+id/titleTV"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:layout_marginStart="10dp"  
    android:layout_marginTop="10dp"  
    android:text="TextView"  
    app:layout_constraintStart_toEndOf="@+id/iconIV"  
    app:layout_constraintTop_toTopOf="@+id/iconIV" />  
</androidx.constraintlayout.widget.ConstraintLayout>
```

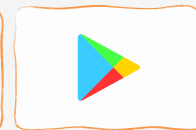




Example 08.2: Add icons and Create data

```
class MainActivity : AppCompatActivity() {  
    var maintitles = arrayListOf<Int>()  
    var subtitles = arrayListOf<Int>()  
    var icons = arrayListOf<Int>()  
  
    fun createData(){  
        maintitles = arrayOf("Charmender", "Togepi", "Squirtle", "Bulbasaur")  
        subtitles = arrayOf("Có thể tiến hóa thành Charmeleon",  
            "Có thể tiến hóa thành Lugia",  
            "Có thể tiến hóa thành Wartortle", "Có thể tiến hóa thành Venusaur")  
        icons = arrayListOf(R.drawable.pokemon1, R.drawable.pokemon2,  
            R.drawable.pokemon3, R.drawable.pokemon4)  
    }  
}
```





Example 08.2: Create my adapter

```
class MyListAdapter  
(  
    private val context: Activity, private val maintitles: List<String>,  
    private val subtitles: List<String>, private val icons: List<Int>  
) : ArrayAdapter<String>(context, R.layout.hd_list_item1, maintitles) {
```

create constructor and
extend ArrayAdapter
class

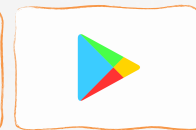
```
    override fun getView(position: Int, convertView: View?, parent: ViewGroup): View {  
        val inflater = context.layoutInflater  
        val rowView: View = inflater.inflate(R.layout.hd_list_item1, null, true)  
        val titleText = rowView.findViewById(R.id.titleTV) as TextView  
        val imageView: ImageView = rowView.findViewById(R.id.iconIV) as ImageView  
        val subtitleText = rowView.findViewById(R.id.subtitleTV) as TextView
```

```
        titleText.text = maintitles[position]  
        imageView.setImageResource(icons[position])  
        subtitleText.text = subtitles[position]
```

```
        return rowView
```

```
    }  
}
```

override for returning
your custom row



Example 08.2:

MainActivity

```
var selectedTV: TextView? = null
var customListView: ListView? = null

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    customListView = findViewById(R.id.customListView)
    selectedTV = findViewById(R.id.selectedTV)

    createData()

    val adapter = MyListAdapter(this, maintitles, subtitles, icons)
    customListView!!.adapter = adapter

    customListView!!.setOnClickListener { adapterView, view, i, l ->
        val text = "Position: " + i.toString() + "\nTitle: " + maintitles[i]
        selectedTV!!.setText(text)
    }
}
```